

Zela+ — Relatório de Status do Sistema

Data: 05 de outubro de 2026 Trabalho final de pós-graduação em Agentes de IA

Este relatório documenta o estado atual do Zela+ a partir da validação manual de ponta a ponta feita sobre o sistema já implementado — o que funciona, como está organizado (arquitetura e infraestrutura) e quais problemas reais foram encontrados e corrigidos no processo. Complementa (não substitui) o RELATORIO.md final do trabalho.

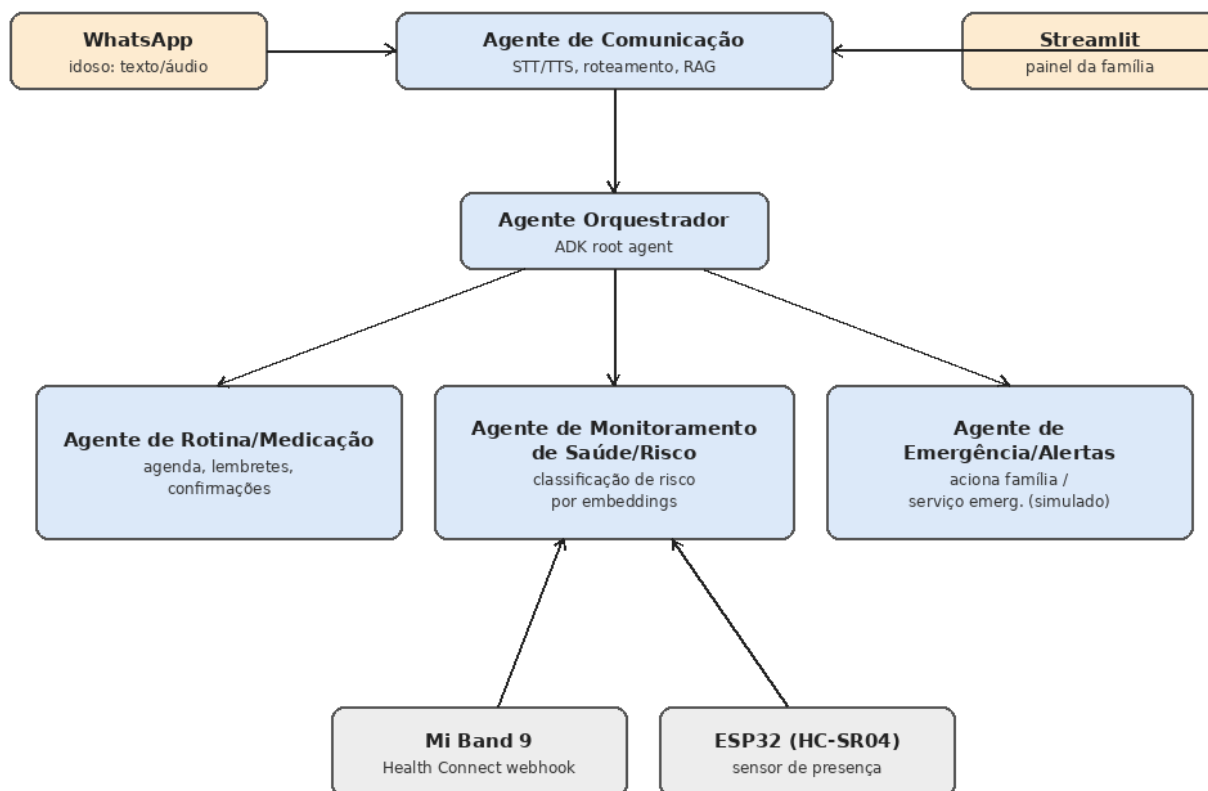
1. Visão geral

Zela+ é um agente de IA multiagente (Google ADK) que ajuda idosos que moram sozinhos a não esquecer medicamentos e compromissos, monitora sinais de risco (saúde e presença/movimento) via smartwatch e ESP32, conversa pelo WhatsApp, mantém a família informada por um painel web, e pode escalar uma situação de risco até simular contato com um serviço de emergência.

2. Arquitetura do sistema

O sistema é dividido em 5 agentes especializados, cada um com responsabilidade única e ferramentas (tools ADK) próprias, orquestrados por um agente raiz:

Arquitetura multiagente do Zela+

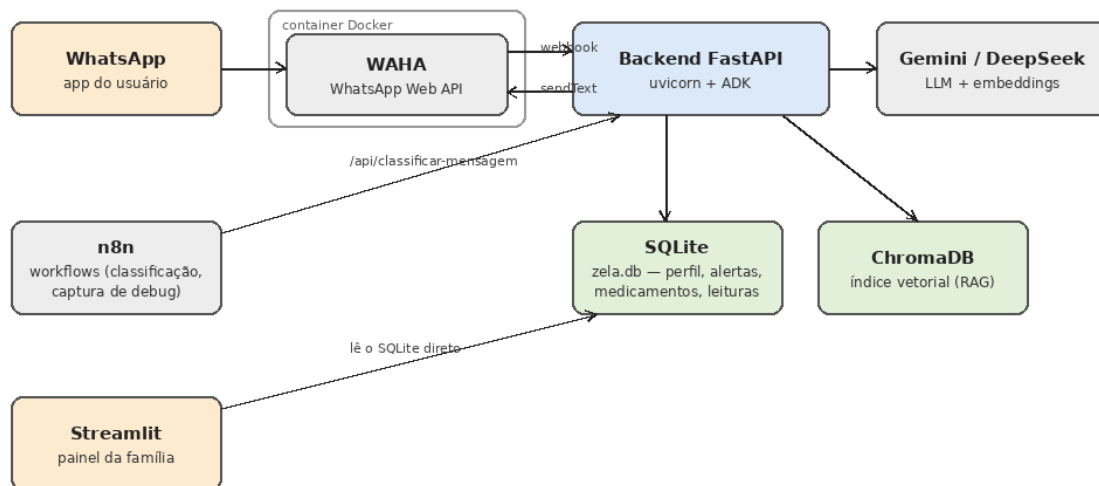


Decisão de escalar risco é sempre determinística (zela/domain/emergencia.py),
nunca tomada diretamente por um LLM — os agentes só alimentam essa máquina de estados.

Decisão de design mais importante do projeto: a decisão de escalar uma emergência (avisar a família, simular contato de emergência) nunca é tomada por um LLM. Ela é calculada por uma máquina de estados determinística e pura (zela/domain/emergencia.py + zela/storage/escalamento.py), coberta por testes automatizados. Os agentes de IA — inclusive o classificador de urgência por embeddings — apenas alimentam essa máquina com eventos; a decisão de agir sobre um evento de risco é sempre determinística e auditável, nunca uma alucinação do modelo.

3. Infraestrutura e implantação

Infraestrutura e implantação



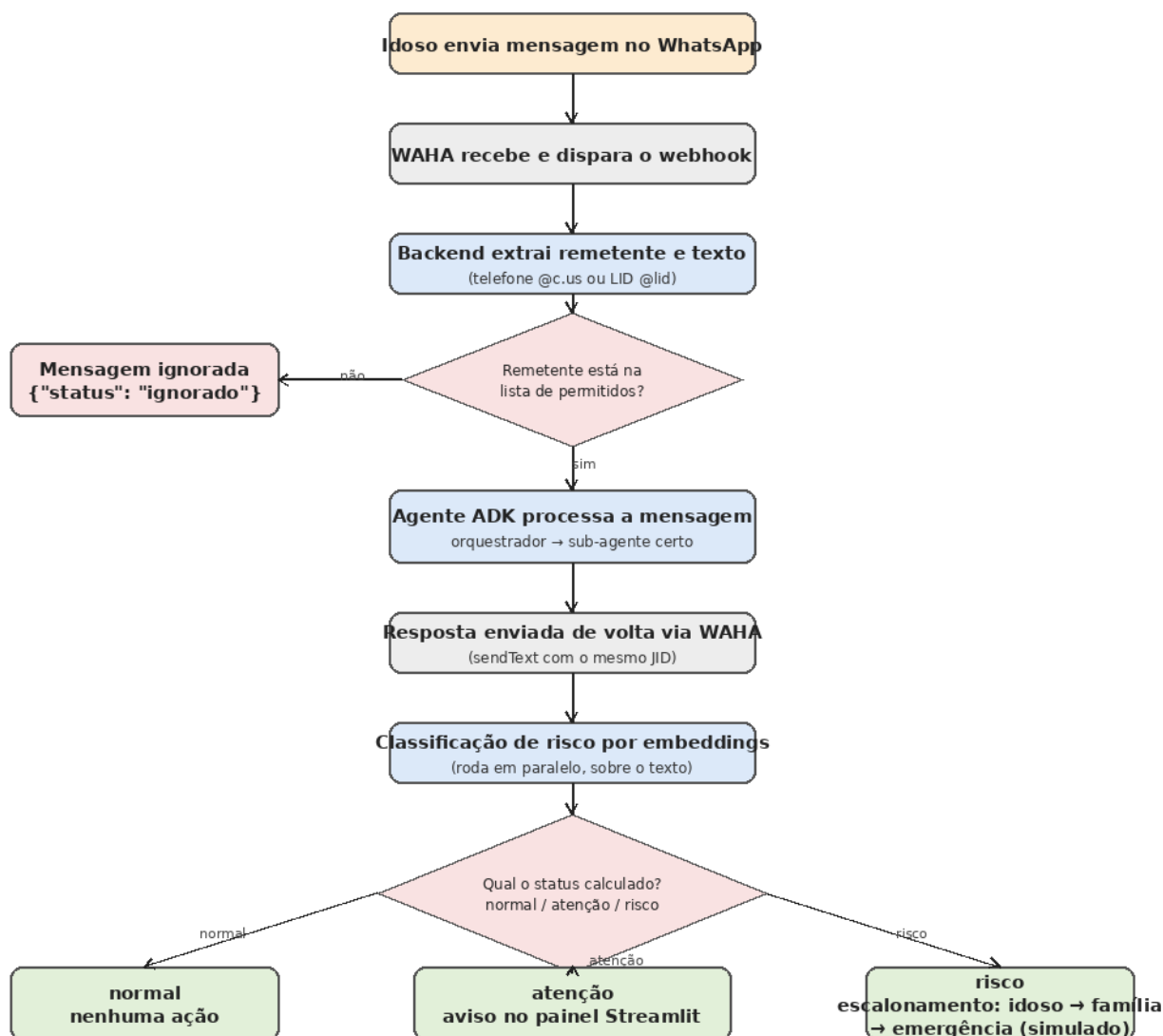
ESP32 (HC-SR04) e Health Connect/Mi Band 9 enviam leituras direto ao Backend (/ingest/esp32, /ingest/health-connect) — ver diagrama de arquitetura de agentes. IPs/portas exatos (gateway do bridge Docker, 0.0.0.0:8000 etc.) variam por ambiente, ver README, seção "Configurando o WAHA (WhatsApp)".

Pontos relevantes da implantação atual:

- WAHA (WhatsApp HTTP API) roda em um container Docker separado, expondo a API REST do WhatsApp Web e um dashboard de administração.
- O backend FastAPI (ADK + lógica de domínio) roda fora do Docker (uvicorn), e precisa escutar em 0.0.0.0 — não só 127.0.0.1 — para ser alcançável pelo container do WAHA através do gateway da rede bridge do Docker (não existe `host.docker.internal` em Docker nativo no Linux, diferente de Docker Desktop).
- Todo estado persistente simples (perfil, medicamentos, confirmações, alertas, leituras de sensores) vive em um único arquivo SQLite (`zela.db`); a busca semântica (RAG sobre bulas e histórico de mensagens) usa um índice ChromaDB separado.
- O LLM de orquestração é Gemini por padrão, com fallback opcional para DeepSeek (via LiteLLM) controlado pela variável `ZELA_LLM_PROVEDOR` — útil quando a cota do Gemini se esgota. Embeddings continuam sempre via Gemini, independentemente do provedor escolhido para o LLM principal.
- n8n roda como uma ferramenta auxiliar separada (não está no caminho crítico de resposta ao usuário): hoje hospeda um workflow de classificação de mensagens (chama `/api/classificar-mensagem`) e um workflow de captura/depuração de payloads do WAHA.
- O painel Streamlit não passa pelo backend — ele lê o `zela.db` diretamente, o que o torna simples mas também significa que não há uma camada de autenticação/API entre o painel e os dados.

4. Fluxo de uma mensagem do WhatsApp, ponta a ponta

Fluxo de uma mensagem do WhatsApp (ponta a ponta)



Este é o fluxo mais crítico do sistema e o que recebeu validação manual mais aprofundada nesta fase — incluindo com mensagens reais de WhatsApp, não só payloads sintéticos de teste.

5. O que foi implementado até agora

Fase	Entregável	Status
Plano 1 — Fundação	Modelos Pydantic + lógica de domínio dos 5 agentes + esqueleto de orquestração ADK, 100% testável sem hardware/APIs externas	Completo

Fase	Entregável	Status
Plano 2 — Comunicação real	Integração com WhatsApp via WAHA	Completo (áudio/STT-TTS ainda não implementado)
Plano 3 — Ingestão de sensores	ESP32 (presença) + Health Connect/Mi Band 9 (saúde)	Completo
Plano 4 — Embeddings	Classificação de urgência por similaridade + RAG sobre bulas/histórico	Completo
Plano 5 — Painel Streamlit	Painel para a família acompanhar medicação, status, alertas e atividade	Completo
Plano 6 — Workflow n8n	Fluxo visual de classificação de mensagem (substitui Langflow, citado no enunciado original — o curso migrou de ferramenta)	Completo
Plano 7 — Empacotamento final	Documentação, relatório, vídeo pitch	Completo

Além dos planos originais, esta fase de validação manual produziu duas correções de bugs reais (detalhadas na seção 7) e suporte opcional a um segundo provedor de LLM (DeepSeek), não previsto no escopo original.

6. Como está funcionando — resultado da validação manual

Diferente da suíte de 200 testes automatizados (que usa dublês/payloads sintéticos e não toca em LLMs, bancos externos ou hardware reais), esta etapa validou o sistema de verdade, subsistema por subsistema:

- Orquestração de agentes (adk run): confirmado que o roteamento entre orquestrador e sub-agentes funciona corretamente com o prefixo de contexto ([contexto do sistema: idoso_id=...; agora=...]), tanto com Gemini quanto com DeepSeek como LLM.
- Embeddings/RAG: reindexação no boot da API confirmada sem erros após a correção do modelo descontinuado.
- Backend (API REST): endpoint de classificação de mensagem (/api/classificar-mensagem) testado e retornando status/motivo corretos para os três níveis de urgência.
- Painel Streamlit: as 4 seções principais (medicação do dia, status atual, histórico de alertas, atividade/presença) foram validadas com dados reais semeados no banco, cada uma refletindo corretamente o estado do domínio.
- Workflow n8n: workflow de classificação executado de ponta a ponta pela interface do n8n, incluindo os três ramos de resposta (risco, atenção, normal) — uma particularidade de exibição do painel do nó "Respond to Webhook" (mostra o dado de entrada, não a resposta HTTP real) foi investigada e confirmada como comportamento esperado da ferramenta, não um bug do projeto.

- WAHA/WhatsApp: sessão conectada de verdade via leitura de QR code pelo dashboard do WAHA; mensagens reais enviadas pelo WhatsApp chegam ao backend. O caminho de resposta (backend → WAHA → WhatsApp) está implementado e corrigido (ver seção 7), mas seu teste de ponta a ponta real ainda está bloqueado por uma chave de API do WAHA ainda não localizada (ver seção 8).

7. Problemas reais encontrados e corrigidos

Cada um destes só ficou visível depois que o anterior foi corrigido e o fluxo avançou um passo adiante — típico de depuração de integração real contra sistemas externos (WhatsApp, WAHA, Gemini) que a suíte de testes automatizados, por desenho, não exercita.

#	Problema	Causa raiz	Correção	Status
1	404 NOT_FOUNDno modelo Gemini do orquestrador	Modelo <code>gemini-2.0-flash</code> descontinuado pela API	Trocado para <code>gemini-3.8-flash</code>	Corrigido
2	404 NOT_FOUNDao gerar embeddings no boot	Modelo <code>text-embedding-004</code> descontinuado	Trocado para <code>gemini-embedding-2</code>	Corrigido
3	Painel do n8n parece mostrar resposta errada no nó "Respond to Webhook"	Particularidade documentada no próprio código do nó: o painel sempre exibe o dado de entrada, não o HTTP de fato enviado	Não é bug — confirmado via <code>curl -i</code> direto no webhook ativo	Comportamento esperado, documentado
4	Mensagem real do WhatsApp nunca gerava resposta; remetente chegava como " <code><id>@lid</code> " em vez de telefone	Recurso de privacidade do WhatsApp (LID): o número real deixou de ser revelado para certos remetentes	Campo <code>lid_whatsapp</code> cadastrado manualmente no perfil; webhook e cliente WAHA passaram a tratar <code>@lid</code> como identificador válido, preservando o comportamento para telefones normais	Corrigido

#	Problema	Causa raiz	Correção	Status
5	Com o LID corrigido, o agente respondia mas o envio de volta falhava com 401 Unauthorized	O cliente WAHA do projeto nunca enviava nenhum cabeçalho de autenticação; esta instância do WAHA exige X-API-Key	Cliente WAHA ganhou suporte a <code>api_key</code> (header <code>X-API-Key</code>), lido de uma variável de ambiente nova	Código corrigido, chave real ainda não localizada

8. Status atual e próximos passos

O sistema está funcionalmente completo e testado (200 testes automatizados passando, cobrindo toda a lógica de domínio, persistência e orquestração). A validação manual de ponta a ponta confirmou que todos os subsistemas funcionam isoladamente e que a maior parte do caminho WhatsApp → agente → resposta já foi comprovada com mensagens reais.

Bloqueio atual: o envio da resposta de volta pelo WhatsApp depende de uma chave de API do WAHA que ainda não foi localizada — o dashboard desta versão do WAHA não expõe esse valor de forma óbvia, e tentativas de adivinhar a chave via requisições diretas à API não tiveram sucesso.

Próximos passos:

1. Localizar a chave de API real do WAHA (inspecionando a variável de ambiente do container diretamente, ou recriando o container com uma chave conhecida — ação que requer atenção porque afeta a sessão do WhatsApp já conectada).
2. Validar o envio de resposta de ponta a ponta com uma mensagem real.
3. Capturar as capturas de tela pendentes (painel Streamlit, workflow n8n, dashboard do WAHA) para o relatório final.
4. Regravar o vídeo pitch com o fluxo completo funcionando.